



307307

Generative AI

Introduction to Agentic AI

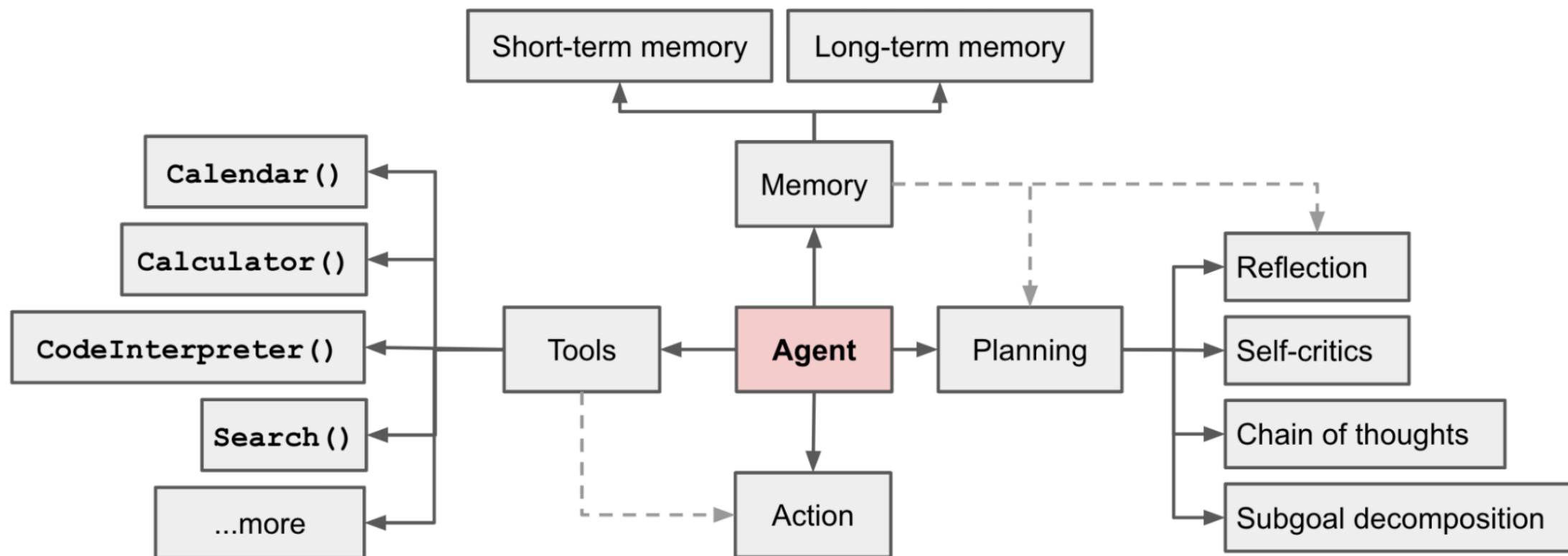
Introduction to Agentic AI

- Large Language Models (LLMs) are very good at producing text: answering questions, summarizing, drafting, and explaining. But on their own, they behave like a “single interaction” system: you ask once, they answer once.
- Many real tasks do not fit that shape. For example:
 - **Up-to-date facts** (the model may not know what happened yesterday).
 - **Precise computation** (the model can make arithmetic or logic mistakes).
 - **Verification** (the model cannot reliably prove it is correct).
 - **Multi-step coordination** (the model may skip steps or lose track).
 - **Interaction with systems** (files, APIs, databases, code, tickets, emails).
- These tasks require more than writing text. They require **doing steps, checking results**, and sometimes **using external systems** (like search engines, files, APIs, or code execution).
- **Agentic AI** is the approach of building AI that can handle this kind of work reliably: not only generating language, but also managing a multi-step process toward a goal.

What We Mean by “An Agent” (Plain Definition)

- An **agent** is an AI system that is **given a goal** and can make progress toward that goal by deciding what to do next (Planning).
- In simple terms, an agent is like an assistant that can:
 - read your instruction,
 - decide the next step,
 - and continue step-by-step until it reaches a usable result.
- The difference from a normal chatbot is not “intelligence.” The difference is **behavior over time**:
 - A chatbot gives an answer and stops.
 - An agent keeps working until the goal is achieved, or it decides it cannot proceed.
- This does not mean the agent is “independent” or “uncontrolled.” In well-designed systems, the agent operates inside boundaries: limited permissions, limited tools, and clear stopping conditions.

Agent Architecture



LLM Agent Architecture: Big Picture

Core idea: An **LLM agent** is an AI system where a **Large Language Model** acts as the central reasoning engine. It does not only generate text; it can also plan, use tools, remember context, and take actions.

Main components:

- **LLM Agent**
- The central controller powered by a Large Language Model
- Understands the user's request
- Decides what information or tool is needed
- Generates the final response

Memory

- Helps the agent keep track of context
- Includes:
 - **Short-term memory:** current conversation and immediate task details
 - **Long-term memory:** stored user preferences, previous interactions, or persistent knowledge

LLM Agent Architecture: Big Picture

Planning

- The agent breaks a task into steps before answering
- May involve:
 - Reflection
 - Self-critique
 - Subgoal decomposition
 - Reasoning over possible actions

Tools

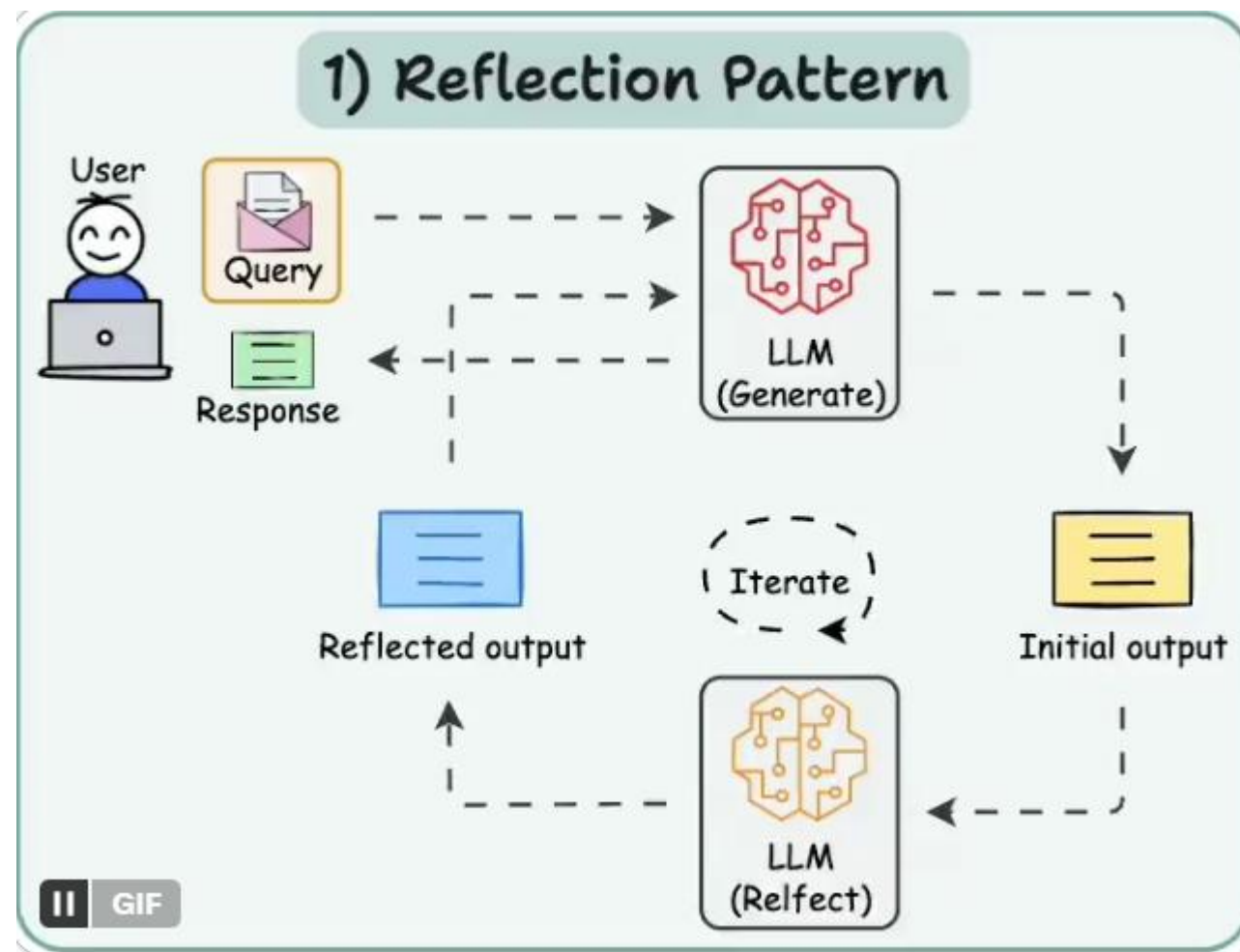
- External systems the LLM can call
- Examples:
 - Search
 - Calculator
 - Calendar
 - Code interpreter
 - Weather API
 - Databases

Action

- The agent executes the selected step
- This could mean calling a tool, retrieving information, updating a calendar, or producing an answer

Planning Capabilities in an LLM Agent

- **Planning** is the part of an LLM agent that helps it decide **how to solve a task**, not just what answer to generate. It allows the agent to break a request into steps, evaluate possible actions, check its own work, and improve the final response.
- **Reflection** means the agent reviews what it has already done and checks whether the result is complete and useful. For example, after answering a weather question, it may check whether it included the city, current temperature, and weather condition.
- **Self-critique** means the agent looks for possible mistakes or weak assumptions in its own response. For example, if the user asks about the temperature in Amman, the agent recognizes that weather changes constantly and should not be answered from memory.



Planning Capabilities in an LLM Agent

- **Subgoal decomposition** means the agent divides a larger task into smaller steps. For example, to answer “**What is the temperature in Amman?**”, the agent identifies the location, determines that live weather data is needed, calls a weather tool, extracts the temperature, and prepares a clear answer.
- **Reasoning over possible actions** means the agent compares different choices before acting. It may consider answering directly, asking for clarification, searching the web, or using a weather tool. In this example, the best action is to use a weather tool because the information is time-sensitive.
- **Takeaway:** These capabilities make the LLM agent more reliable because it can plan, evaluate, correct itself, and choose the right action before giving a final answer.

How the LLM Agent Works

Typical flow:

- **User asks a question or gives a task**
Example: “What is the temperature in Amman?”
- **The LLM interprets the request**
The agent understands that the user wants current weather information.
- **The agent checks whether memory is enough**
Since temperature changes constantly, memory alone is not reliable.
- **The agent plans the next step**
It decides that it needs an external weather tool or search tool.
- **The agent calls a tool**
For example, a weather API is used to get the latest temperature.
- **The tool returns data**
Example: current temperature, weather condition, humidity, wind speed.
- **The LLM formats the answer**
The agent converts the raw tool output into a clear response for the user.
- **Key takeaway:**
The LLM provides reasoning and language understanding, while tools provide accurate, real-time, or specialized information.

Example: Asking About the Temperature in Amman

User request: “What is the temperature in Amman?”

Step-by-step agent behavior:

1. Understand the task

The LLM identifies that the user is asking for the current temperature in **Amman, Jordan**.

2. Decide if a tool is needed

The agent knows that weather data changes frequently, so it should not rely only on its stored knowledge.

3. Select the right tool

The agent chooses a weather tool or search tool.

4. Call the tool

The tool retrieves live weather data for Amman.

5. Prepare the respond and send it to the user

After the LLM receives the result of the tool, it prepares the final response to the user

Basic Python Example

Code: Setup + Agent Decision

```
import pandas as pd
import google.generativeai as genai
import json

# Load dataset
df = pd.read_csv("sales.csv")    # Example dataset

# Configure Gemini
genai.configure(api_key="YOUR_API_KEY")
model = genai.GenerativeModel("gemini-1.5-flash")

# Tools the agent can use
def summary_tool(_):
    return df.describe().to_string()

def correlation_tool(_):
    return df.corr(numeric_only=True).to_string()

tools = {
    "summary": summary_tool,
    "correlation": correlation_tool
}
```

Code: Setup + Agent Decision

```
# Agent decides next action
def agent_decide(state):
    prompt = f"""
    You are a data analyst agent. Based on the task and dataset preview below,
    choose the best next analytical action.

    Return JSON only:
    {{ "action": "..."}}

    State:
    {state} """
    return model.generate_content(prompt).text
```

Code: Execute Tool + Final Interpretation

```
def execute(action_json):  
    data = json.loads(action_json)  
    action = data["action"]  
  
    if action in tools:  
        return tools[action](None)  
    else:  
        return "Unknown action"  
  
# Initial state includes task + preview of data  
state = f"""  
Goal: Provide insights about the sales dataset.  
Data preview:  
{df.head().to_string()}  
"""
```

Code: Execute Tool + Final Interpretation

```
# Step 1: Agent chooses action
decision = agent_decide(state)
print("Model decision:", decision)

# Step 2: Python executes tool
observation = execute(decision)

# Update state
state += f"\nExecuted: {decision}\nObservation:\n{observation}"

# Step 3: Agent provides final insights
final_prompt = f"""
You are a senior data analyst.
Based on the updated state, produce a short, clear explanation of the insights.
"""

final_answer = model.generate_content(final_prompt).text

print("\nFinal Insight Report:\n", final_answer)
```